

School of Computer Science and Artificial Intelligence

WEEK-12 Assignment # 23

Program : B. Tech (CSE)

Specialization :AIML

Course Title : Competitive Programming

Course Code : 23CS302PC305

Semester : VI

Academic Session : 2025-2026

Name of Student : B.Thanmai

Enrollment No. : 2303A52126

Batch No. : 33

1.Subset Generation Using Bitmask

Elaborated Scenario:

In combinatorics and computer science, generating all subsets of a set is a common problem.

Suppose a company wants to evaluate all possible combinations of features for a product. Each feature can be either included or excluded. Representing subsets using bit masking allows efficient generation of all possible subsets without recursion. Task:

- Given a set of n elements, generate all possible subsets.
- Use bit masking where each bit represents whether an element is included (1) or excluded (0).
- Print the subsets in lexicographical or input order.

Example Input:

3

A B C

Expected Output:

{}

{A}

{B}

{C}

{A, B}

{A, C} {B,
C}

{A, B, C}

Explanation:

There are $2^n = 8$ possible subsets for a set of 3 elements. Each subset corresponds to a bitmask of length n:

000 → {}

001 → {C} 010

→ {B}

011 → {B, C} 100

→ {A}

101 → {A, C} 110

→ {A, B}

111 → {A, B, C}

Python:

cp.py > ...

```
1  # Step 1: Input
2  n = int(input())
3  arr = input().split()
4
5  # Step 2: Total subsets = 2^n
6  total = 1 << n  # same as 2^n
7
8  # Step 3: Loop through all bitmasks
9  for mask in range(total):
10     subset = []
11
12     # Step 4: Check each bit
13     for j in range(n):
14         if (mask >> j) & 1:
15             subset.append(arr[j])
16
17     # Step 5: Print subset
18     if len(subset) == 0:
19         print("{}")
20     else:
21         print "{" + ", ".join(subset) + "}"
```

```
3
a b c
{}
{a}
Downloads/hpc_log_app_fixed/cp.py
3
a b c
{}
{a}
{b}
{a, b}
3
a b c
{}
{a}
{b}
{a, b}
a b c
{}
{a}
{b}
{a, b}
{}
{a}
{b}
{a, b}
{b}
{a, b}
```

Java:

```
1  import java.util.*;
2
3  public class SubsetBitmask {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          // Step 1: Input
8          int n = sc.nextInt();
9          String[] arr = new String[n];
10
11          for (int i = 0; i < n; i++) {
12              arr[i] = sc.next();
13          }
14
15          // Step 2: Total subsets
16          int total = 1 << n; // 2^n
17
18          // Step 3: Loop through all bitmasks
19          for (int mask = 0; mask < total; mask++) {
20
21              System.out.print("{");
22              boolean first = true;
23
24              // Step 4: Check each bit
25              for (int j = 0; j < n; j++) {
26                  if ((mask & (1 << j)) != 0) {
27
28                      if (!first) {
29                          System.out.print(", ");
30                      }
31
32                      System.out.print(arr[j]);
33                  }
34              }
35
36              System.out.println("}");
37          }
38      }
39  }
```

```
1  import java.util.*;
2
3  public class SubsetBitmask {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          // Step 1: Input
8          int n = sc.nextInt();
9          String[] arr = new String[n];
10
11          for (int i = 0; i < n; i++) {
12              arr[i] = sc.next();
13          }
14
15          // Step 2: Total subsets
16          int total = 1 << n; // 2^n
17
18          // Step 3: Loop through all bitmasks
19          for (int mask = 0; mask < total; mask++) {
20
21              System.out.print("{");
22              boolean first = true;
23
24              // Step 4: Check each bit
25              for (int j = 0; j < n; j++) {
26                  if ((mask & (1 << j)) != 0) {
27
28                      if (!first) {
29                          System.out.print(", ");
30                      }
31
32                      System.out.print(arr[j]);
33                      first = false;
34                  }
35              }
36
37              System.out.println("}");
38          }
39
40          sc.close();
41      }
42  }
```